

AROL Project Q2

TEAM MEMBERS:

ESTEBAN PUYO

FELIPE ROJAS

FERNANDO VELILLA

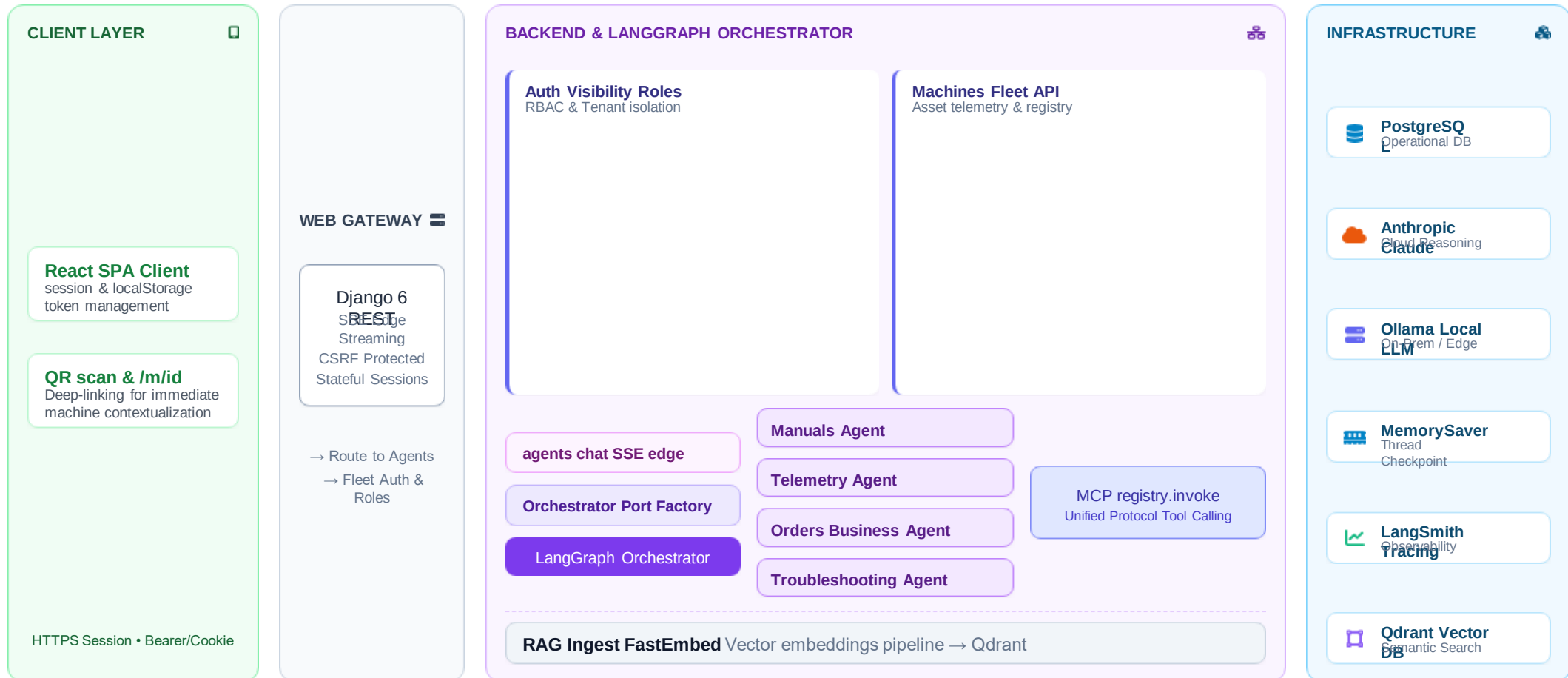
Objectives

- Build a conversational chatbot that assists plant operators through a specialized multi-agent ecosystem
- Agents capable of reasoning over heterogeneous data-sources
- Data sources include:
 - Technical documentation
 - Machine telemetry
 - Alarm history
 - Maintenance records
 - Commercial and contractual information

Developments

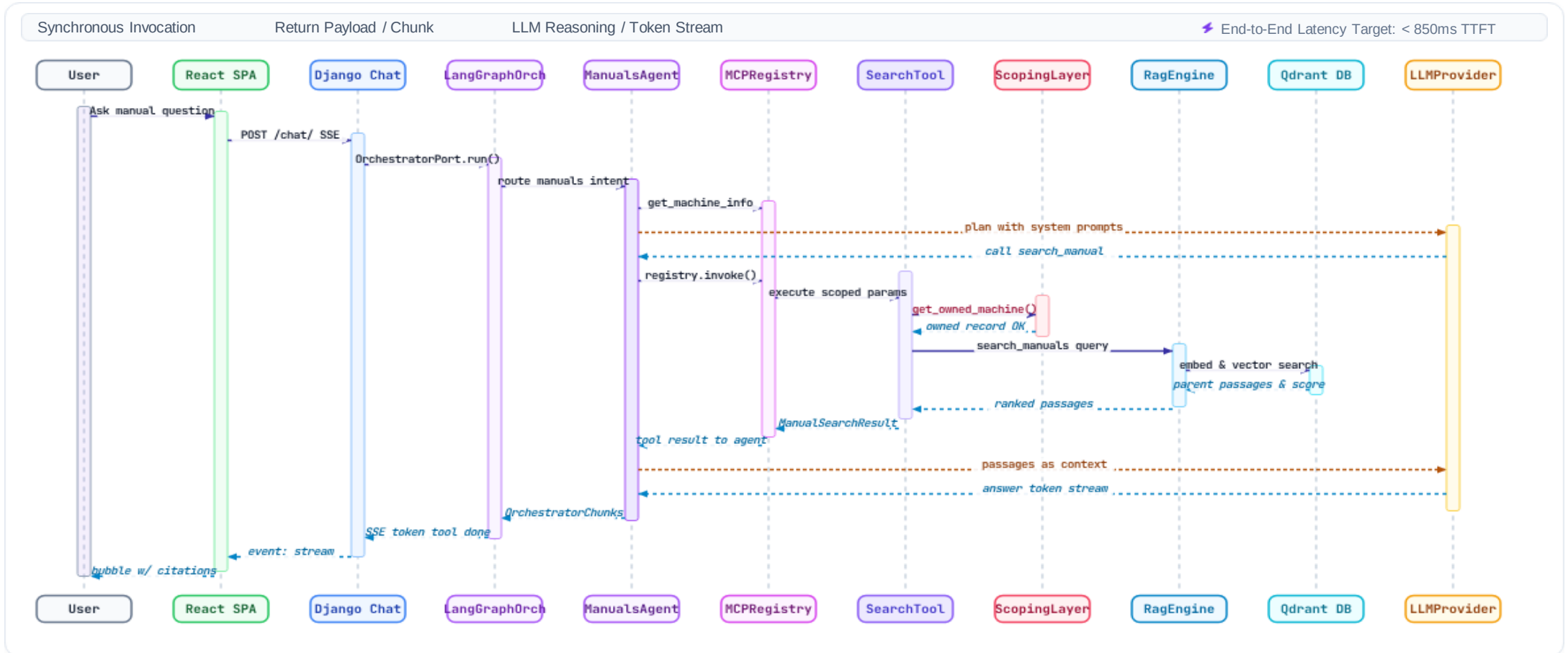
- Full-stack web application with an interactive UI including the chatbot, basic machine information, PDF viewer with manual and relevant data for the type of user (maintenance records or commercial data)
- Authentication and visibility layer: user is restricted to specific data depending on the category and company
- Chatbot with persistence, memory and allows to have multiple conversations
- Orchestrator and multi-agent solution based on LangGraph
- RAG engine: implemented a vector DB with Qdrant that allows fast queries over PDF manuals using vector embeddings
- LLM model comparison: Two models were benchmarked, one with API costs (claude haiku) and a local model (qwen 2.5)

Conceptual Architecture



End-to-end request flow

Manual search process



Tech stack used

Frameworks, tools, databases that we used



Backend:

Python, Django 6,
MCP tool server,
LangGraph orchestrator



Frontend:

React 19,
TypeScript, Vite 8



Relational DB:

PostgreSQL



Vector DB: Qdrant +
FastEmbed(RAG)



AI layer:

MCP tools (Pydantic
schemas),
SSE chat API,
Anthropic/Ollama LLMs



Authentication:

Django session-based,
Role-based visibility,
MCP session cookies

Containerized Deployment

With Docker Compose



USER BROWSER

SPA Client
localhost:8080
React Static Build

Reverse Proxy
nginx frontend
container port 80
proxy_pass /api

Compose Services Network bridge / internal

Core Engine
Django backend
uvicorn 8000 (ASGI)
LLM_PROVIDER anthropic
LANGSMITH tracing
OLLAMA_URL host.gateway

Vector DB
qdrant v1
profile: local qdrant
port 6333 / gRPC

RAG Pipeline
ingest markdown manuals CLI
Batch Vector Ingestion Tool

Relational DB
postgres 16 alpine db
port 5432 internal

VOLUMES & MOUNTS

- qdrant_data volume**
- postgres_data volume**
- fastembed_cache volume**
- Data Manuals .md ro bind**

EXTERNAL ENDPOINTS

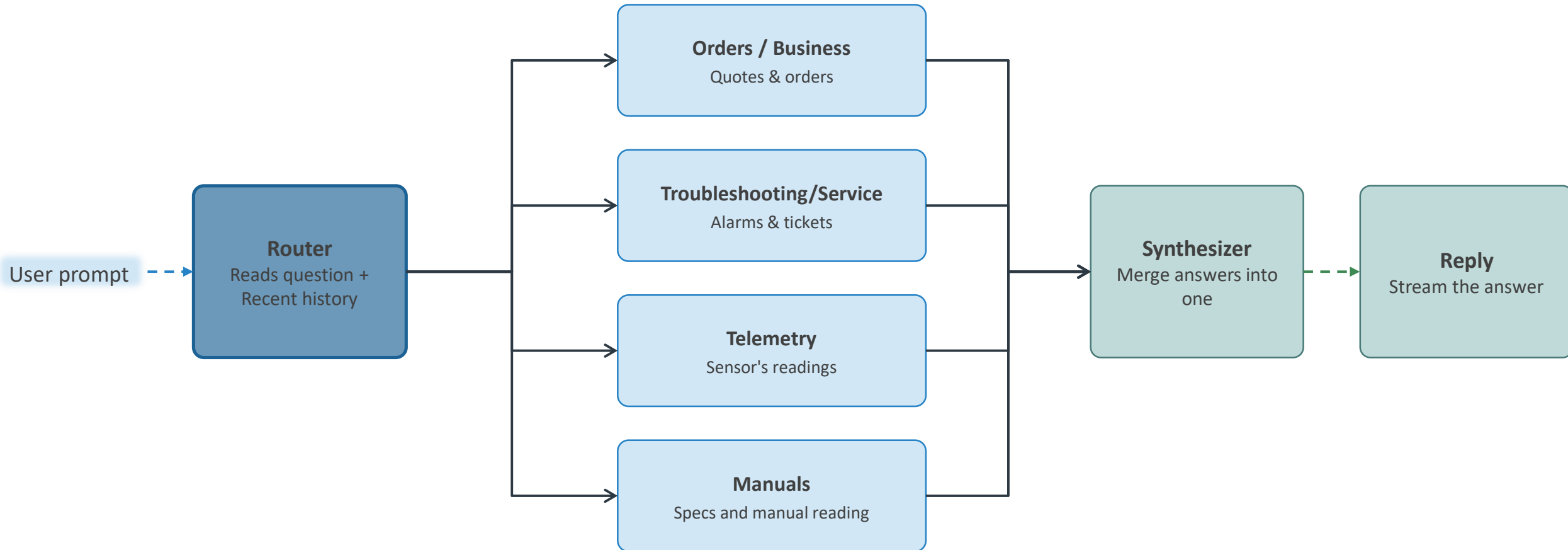
- Ollama host port 11434**
- Anthropic Cloud API**
- Qdrant Cloud cluster**

Docker Compose Topology

Container / Service	Base Image / Port	Volume / Mount Target	Environment / Variables
Nginx Frontend	nginx:alpine • Port 80	Static React SPA bundle	Proxies /api/* to Django Uvicorn
Django Backend	python:3.12-slim • Port 8000	fastembed_cache, data_manually	LLM_PROVIDER=anthropic, LANGSMITH=true
Qdrant Vector DB	qdrant/qdrant:v1 • Port 6333	qdrant_data persistent volume	Supports local profile or hybrid cloud cluster
Postgres Database	postgres:16-alpine • Port 5432	postgres_data persistent volume	ACID store for telemetry, users, and machine registry
Manuals Ingest CLI	python:3.12-slim • CLI job	./manuals/*.md (read-only bind)	Batch embeds and pushes vectors to Qdrant collection
Ollama Edge Host	Host IPC / GPU • Port 11434	Local model weights (GGUF)	OLLAMA_URL=http://host.docker.internal:11434

Orchestrator

Reads the question or prompt received and decides which "specialists" are needed



Orchestrator

Important features and choices that make a better product



Instead of forcing one topic per turn, asks the four specialists to combine the answer into a great one.



It can read the history of the chat and keeps the conversation running without losing context



When only one specialist is needed it skips the extra merge step entirely saving money and reducing latency



Each conversation is specifically separated by type of user, and by machine.

Agents

What's behind the agents? How each of them works

Characteristics



The prompt

A short set of instructions: what it covers, what it must refuse, and the most important, always look up real data, never guess.



Its tools

A small, fixed toolbox which agents can only use them



Its loop

It reads the task, calls a tool, reads the result, and repeats until it can answer, then writes the reply.

Pipeline



Read the task

The coordinator's clear instruction



Call a tool

Fetch real data from the data layer



Read the result

Structured, verified facts



Write the answer

Citing IDs, codes, page numbers

Guardrail: If no real data available, no answer. Agents won't invent codes, prices, or steps.

Agents

What each agent does and which tools they call

Specialist	What it answers	Tools it can call
 Orders / Business	Quote history & revisions · order status	look up quotes · check order status
 Troubleshooting / Service	Diagnose alarms & faults · open / find tickets	search error codes · list alarms · list tickets · open ticket
 Telemetry	Historical sensor metrics & operating status	query telemetry
 Manuals	Operating & maintenance guidance · product specs	search manual (documents)

Agents

These are the tools and actions the agents can make



Orders / Business

- **get_quote_history** A quote and all its revisions — prices, status, accept/reject history.
- **get_order_status** A confirmed order's status and amount, linked to its quote.



Troubleshooting / Service

- **list_alarms** Real alarm events on the machine — code, severity, status, time.
- **search_error_codes** What a code or symptom means, plus the recommended action.
- **list_maintenance_tickets** Service tickets — type, status, priority, owner, linked alarm.
- **create_ticket** Opens a new field-support ticket for the machine.



Telemetry

- **query_telemetry** Recent sensor readings for a metric available, each tagged with operating status. Aggregates for better precision



Manuals

- **search_manual** The most relevant manual passages for a how-to do something or a maintenance question.

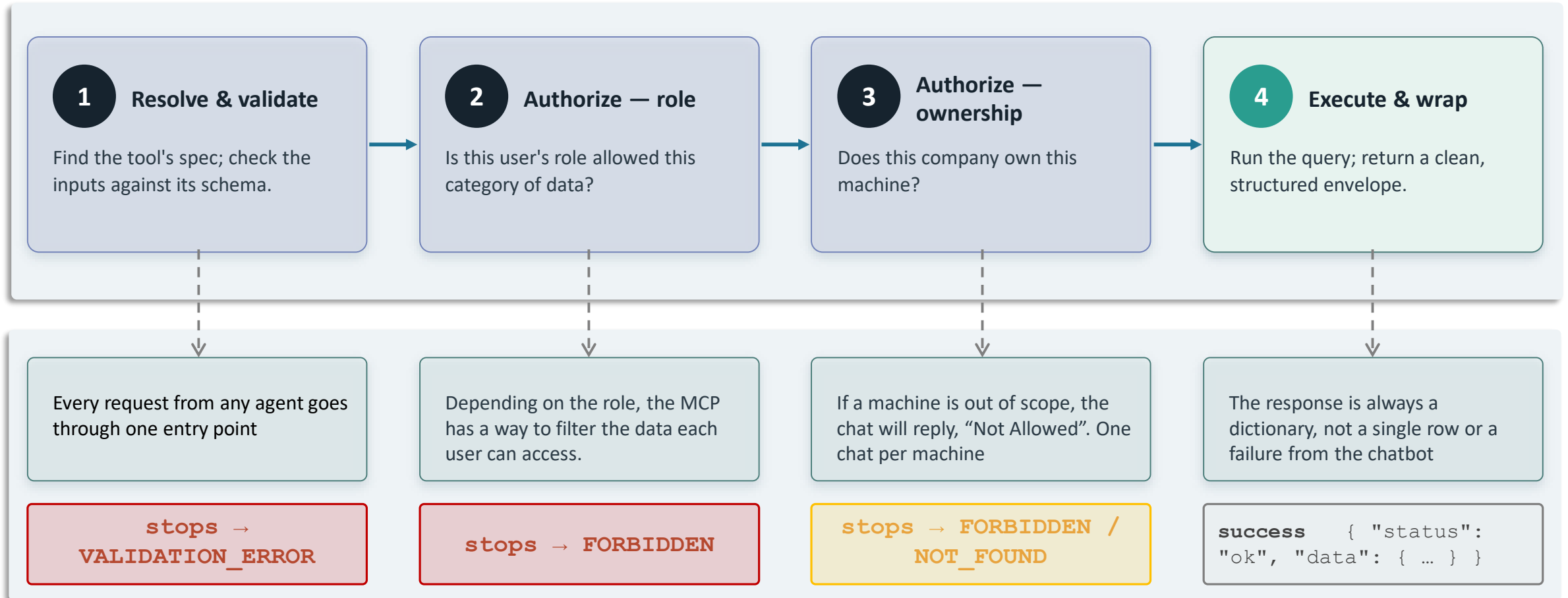


General

- **get_machine_info** Loads the machine record — used to lock every request to this unit and show basic information in the frontend

MCP Server

What it does and what it guarantees



Vector DB

Details of the RAG retrieval feature of the project



Qdrant vector DB: cloud implementation (free tier solution)



Embedding model: FastEmbed (runs locally via ONNX)



Embedding dimensionality: 384 (dense vectors)



Similarity method: Cosine distance



PDF to Markdown conversion: PyMuPDF (0.1-0.5 s/page)



Filtering using machine serial numbers contained in the payload field



Text chunking strategy: Parent-child recursive splitters

Benchmarking

Questions used to benchmark and evaluate the project

Manuals RAG 1: What lubricants can be used for the machine and how to apply them?

Manuals RAG 2: Which safety procedures should be followed before performing maintenance on machine ``<serialNumber>``?

Quotes: What was the latest quotation revision issued to a customer, and how did it change?

Quote lines: List all line items and net prices for the latest approved revision of quotation ``<quoteld>`'. Note which lines link to a specific installed machine vs standalone parts

Orders: Which orders currently have a shipmentStatus of 'In production' or 'Ready for shipment'?

Order lines: What is the current fulfillmentStatus of all order lines under order ``<orderId>``?

Telemetry snapshots 1: What was the average operating temperature, total energy consumption, and minimum uptime percentage for machine ``<serialNumber>`` on 2026-08-04?

Telemetry snapshots 2: What were the last production rate readings from the machine?

Alarms: Why is machine ``<serialNumber>`` generating repeated alarms?

Maintenance tickets 1: What maintenance activities were recently performed on a given machine?

Maintenance tickets+Manuals: What periodic maintenance is due for a given machine, and when is the next one given the last maintenance procedures?

Alarms+Manuals: What does alarm ``AL017_LOW_AIR_PRESSURE`` mean, and what does the manual recommend?

Orders+Quote: When was the last order delivered and how much did it cost?

Results

Results evaluating the models

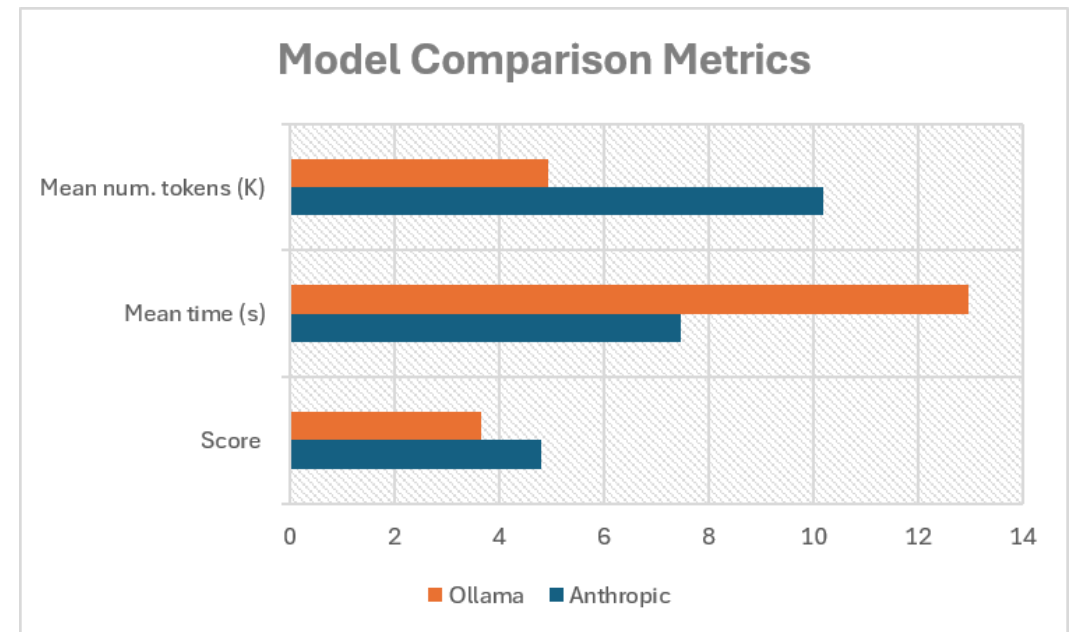
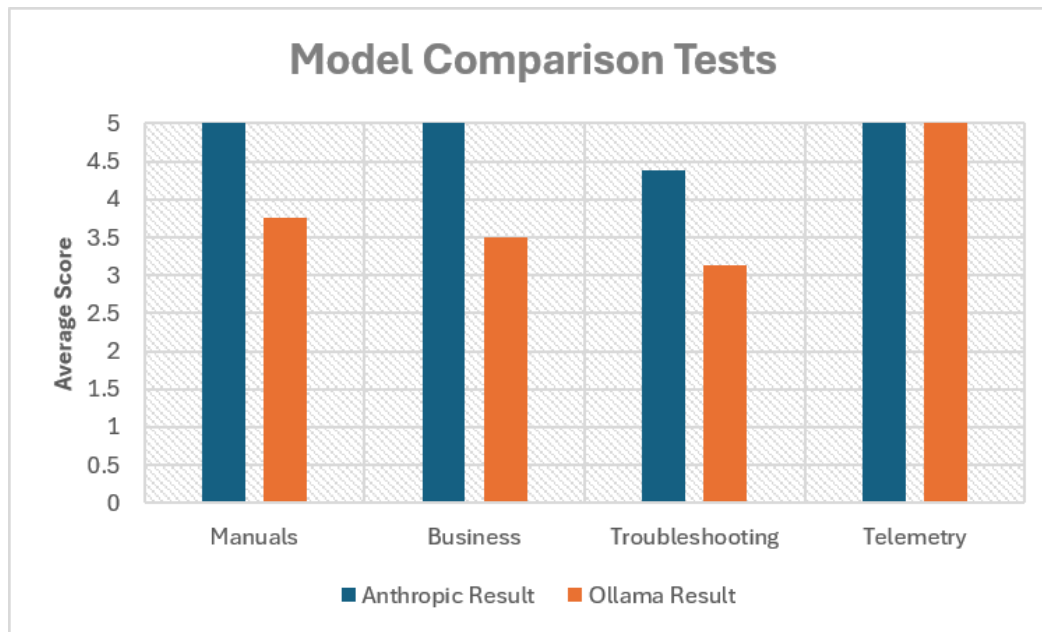
Model response was classified as:

- Successful: model can pick the right data and answer correctly to the question
- Mid-successful: model retrieves the right data but don't clarify completely the question or misses some data
- Failure: model don't pick the right data and thus can't answer the question, or model hallucinates

Test type	Agent	Anthropic Result	Anthropic tokens	Anthropic time (s)	Anthropic cost	Ollama Result	Ollama tokens	Ollama time (s)
Manuals RAG 1	Manuals	Successful	9500	9.01	\$ 0.0119	Mid-successful	3700	19.36
Manuals RAG 2	Manuals	Successful	14600	11.15	\$ 0.0171	Successful	4000	19.46
Quotes	Business	Successful	5600	5.06	\$ 0.0070	Mid-successful	4700	9.62
Quote lines	Business	Successful	5200	5.66	\$ 0.0070	Mid-successful	3000	4.4
Orders	Business	Successful	5000	4.45	\$ 0.0062	Successful	1600	3.28
Order lines	Business	Successful	5500	3.21	\$ 0.0064	Successful	3100	29.2
Telemetry snapshots 1	Telemetry	Successful	9300	7.34	\$ 0.0122	Successful	4700	9.15
Telemetry snapshots 2	Telemetry	Successful	7600	5.57	\$ 0.0094	Successful	5300	3.94
Alarms	Troubleshooting	Successful	7100	8.16	\$ 0.0099	Mid-successful	10600	31.03
Maintenance tickets 1	Troubleshooting	Successful	7100	6.09	\$ 0.0097	Successful	4300	11.55
Maintenance tickets+Manuals	Troubleshooting	Mid-successful	15800	11.59	\$ 0.0192	Failure	5300	8.92
Alarms+Maintenance	Troubleshooting	Successful	32500	13.9	\$ 0.0377	Successful	10200	10.82
Orders+Quote	Business	Successful	7700	5.92	\$ 0.0090	Mid-successful	3800	7.85
Metric value		4.807692308	10192	7.47	\$ 0.0125	3.653846154	4946	12.96769231
Metric type		Score (from 0 to 5)	Mean num. tokens	Meantime (s)	Mean cost/prompt	Score (from 0 to 5)	Mean num. tokens	Meantime (s)

Results

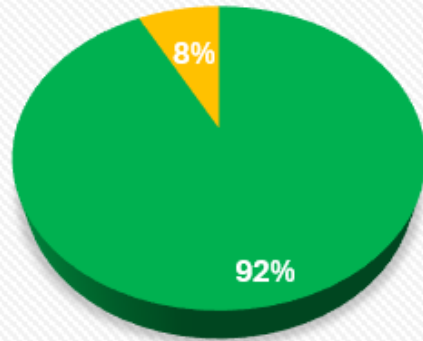
- **Model score comparison:** Score was calculated giving to each test a value of 5 (successful), 2.5 (mid-successful), 0 (failure). Then we obtain the average score among all tests for each model and agent type.



Results

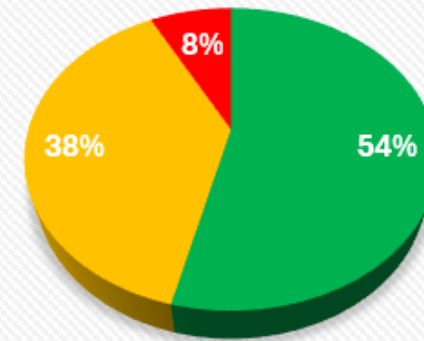
- Success rate for each model:

Anthropic



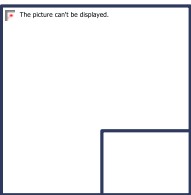
■ Successful ■ Mid-successful ■ Failure

Ollama



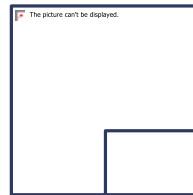
■ Successful ■ Mid-successful ■ Failure

AI models comparison



API models

- **Advantages:**
 - Frontier reasoning capabilities: more intelligence
 - Less prone to model hallucination or making mistakes
 - Excel at tool calling
- **Disadvantages:**
 - Costs: depending on the model a given cost is involved
 - Low privacy: data leaves the company



Local models

- **Advantages:**
 - Fully sovereign and private: information doesn't leave company servers
 - No cost: completely open source
- **Disadvantages:**
 - More prone to model hallucination and returning false information
 - Less reasoning capabilities than frontier models (unless the company servers have plenty of computing power with GPU devices)
 - Chatbot responsiveness: higher latency

Model	Provider	Context	Max Output	Input (\$/1M tokens)	Output (\$/1M tokens)
claude-haiku-4-5	Anthropic	200K	64K	\$1.00	\$5.00
qwen2.5:3b	Ollama	32K	8K	\$0	\$0

Observability



Additional resources to evaluate and monitor agentic workflows

LangSmith (tool compatible with LangGraph) was used to:

- Evaluate orchestrator decision making
- Observe and understand the agentic workflows
- Debug errors in multi-turn conversations
- Analyze tool use of the agent
- Monitor inputs, outputs, token costs and latency from the different models
- Build an improvement loop for the agents, first we observe, identify problems, then modify the architecture, system prompt or tools, then repeat the process

LangSmith config.

The use of LangSmith requires to setup an account, obtain an API key and configure these environment variables:

- `LANGSMITH_TRACING=true`
- `LANGSMITH_ENDPOINT=https://api.smith.langchain.com`
- `LANGSMITH_API_KEY=<api-key>`
- `LANGSMITH_PROJECT=<project-name>`

LangSmith demo

- Trace input/output fields:

The screenshot shows a trace with a 'User' node containing the prompt: 'What's the expected delivery date of the current orders with open status'. Below it is an 'AI' node with the response: 'Based on the order data for your machine (EAGLE-VA-TURRET serial A2055), there is one order with open status: ORD-2026-0007 (converted from quote QTE-2026-0011)'. The response includes a bulleted list of details: Expected Delivery Date: 2026-08-12, Status: In production, Shipment Status: In production, Amount: €30,452.00, and Contents: Guard Interlock upgrade to EN ISO 14119, emergency stop circuit revision, and safety relay replacement. A blue arrow labeled 'Prompt' points to the user input, and a green bracket labeled 'Agent Output' encompasses the AI response.

- Router fields:

The screenshot shows the 'Fields' section of a trace. It lists 'intent: orders_business', 'agent_outputs: []', and 'tasks: 0'. Under 'tasks', there is a subtask for 'agent: orders_business' with the text: 'subtask: What is the expected delivery date for all current orders with open status?'. A red oval highlights this subtask, and a red speech bubble next to it says: 'Agents can run in parallel, each with a subtask'.

- Default view:

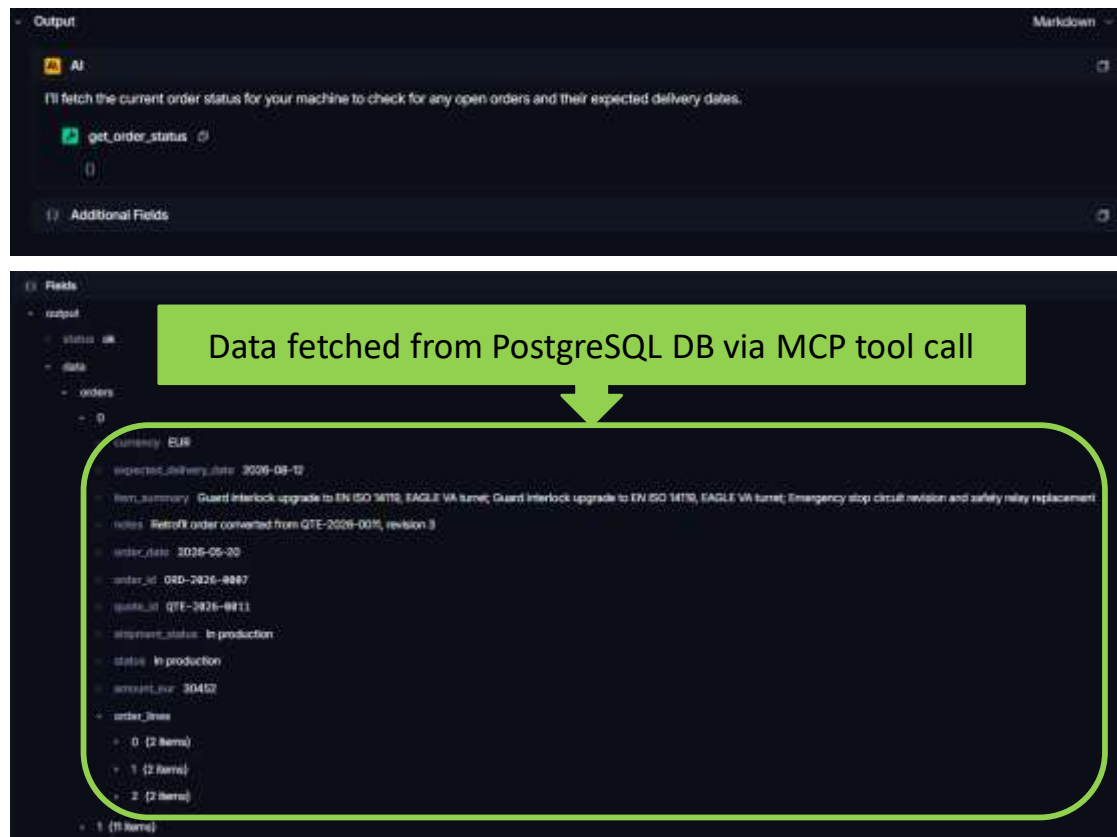
The screenshot shows the default view of a trace. It displays a sequence of nodes: 'router' (1.31s), 'ChatAnthropic' (1.31s), 'PydanticToolsParser' (0.00s), '_dispatch' (0.00s), 'orders_business' (3.23s), 'ChatAnthropic' (0.90s), 'get_order_status' (0.01s), 'ChatAnthropic' (2.29s), and 'synthesizer' (0.00s). The trace is titled 'LangGraph 08/27 10:42 AM' and shows a summary of the run with a cost of \$0.012534.

- Waterfall view:

The screenshot shows the trace in 'Waterfall' view. It visualizes the execution flow of the agents. The 'router' node (1.31s) leads to a 'ChatAnthropic' node (1.31s), which then branches into parallel execution of 'PydanticToolsParser' (0.00s), '_dispatch' (0.00s), 'orders_business' (3.23s), 'ChatAnthropic' (0.90s), 'get_order_status' (0.01s), 'ChatAnthropic' (2.29s), and 'synthesizer' (0.00s). The trace is titled 'LangGraph 08/27 10:42 AM' and shows a summary of the run with a cost of \$0.012534.

LangSmith demo

- Tool call and output fields (get_order_status):

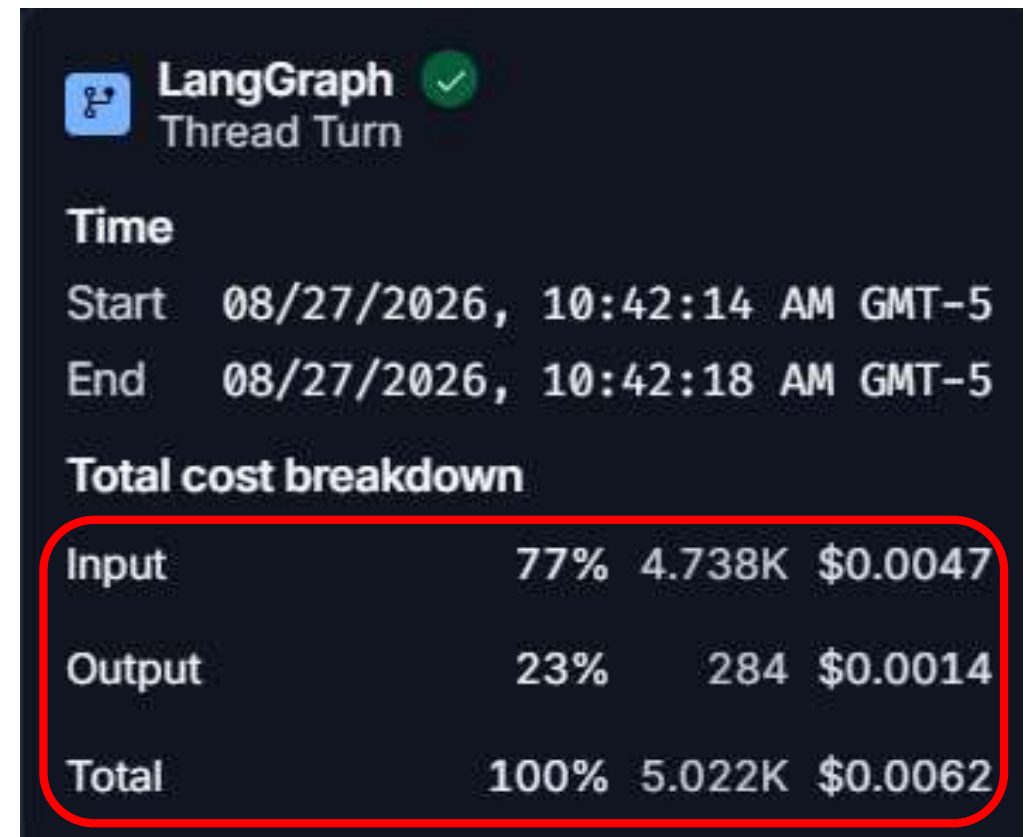


The screenshot shows the LangSmith interface. The top panel displays the tool call 'get_order_status' with an empty parameter list. The bottom panel shows the output fields, which are highlighted by a green box. A green arrow points from a text box 'Data fetched from PostgreSQL DB via MCP tool call' to the output fields.

Data fetched from PostgreSQL DB via MCP tool call

```
{
  "status": "ok",
  "data": {
    "orders": [
      {
        "id": 0,
        "currency": "EUR",
        "expected_delivery_date": "2026-08-12",
        "item_summary": "Guard interlock upgrade to EN ISO 14119, EAGLE VA turret; Guard interlock upgrade to EN ISO 14119, EAGLE VA turret; Emergency stop circuit revision and safety relay replacement",
        "notes": "Retrofit order converted from QTE-2026-0015, revision 3",
        "order_date": "2026-05-20",
        "order_id": "ORD-2026-0007",
        "quote_id": "QTE-2026-0011",
        "shipment_status": "In production",
        "status": "In production",
        "amount_eur": 30452,
        "order_items": [
          {
            "id": 0,
            "name": "0 (2 items)"
          },
          {
            "id": 1,
            "name": "1 (2 items)"
          },
          {
            "id": 2,
            "name": "2 (2 items)"
          }
        ]
      }
    ]
  }
}
```

- Cost breakdown:



The screenshot shows the LangGraph Thread Turn interface. It displays the start and end times of the thread turn and a cost breakdown table. The table is highlighted by a red box.

LangGraph Thread Turn

Time

Start 08/27/2026, 10:42:14 AM GMT-5

End 08/27/2026, 10:42:18 AM GMT-5

Total cost breakdown

Input	77%	4.738K	\$0.0047
Output	23%	284	\$0.0014
Total	100%	5.022K	\$0.0062

App Solution

MACHINE ACCESS

Choose a machine

Scan the QR code on the equipment, or pick a machine assigned to your company.

Scan QR code

Opens the camera scanner in a separate tab so a camera error does not blank the main app. Use paste or upload there if the camera is blocked on HTTP.

TS-EURO-PK-TWIN-CHUTE-D
5/4 15610 - Novara Plant 1 - Bottling Line 3

TSI-EURO-VA-IES-LIGHT
5/4 17203 - Novara Plant 1 - Bottling Line 5

TSI-EURO-VA-IES-LIGHT
5/4 17579 - Alessandria Plant 2 - Bottling Line 1

Ask about your company's fleet

AI: Hi, I'm the AROL fleet assistant. Ask me about the machines your company owns, their models, your company profile, or who else at your company uses this platform.

You: Tell me characteristics about the machines available

AI:

AGENT ACTIVITY

- Listing your company machines...
- list_customer_machines complete list_customer_machines

Here are the characteristics of the machines available to your company:

Machine 1: TS-EURO-PK-TWIN-CHUTE-D

Ask about your machines, models, company or teammates...

General Header

Machine Manual Maintenance tickets **AI Chatbot**

AI Chatbot

AROL customer platform assistant for TSI-EURO-VA-IES-LIGHT · Serial 17203 · USR-002

Chat 1 Chat 2 **Chat 3**

Up to 3 chats at the same time

AI: Hi, I'm the Arol Chatbot assistant for the TSI-EURO-VA-IES-LIGHT (17203). Ask about anything related to the machine, the telemetry, orders and/or services — I'll be happy to help you

You: Show me basic statistics related to the energy consumption of the machine

AI:

TELEMETRY

Which agent is calling

AGENT ACTIVITY

- Routing to the Telemetry agent...
- Loading machine context...
- Machine context loaded get_machine_info
- TSI-EURO-VA-IES-LIGHT
- Querying machine telemetry metrics...
- Telemetry query complete query_telemetry

Here are the basic energy consumption statistics for machine **TSI-EURO-VA-IES-LIGHT (serial 17203)**:

Metric	Value	Unit
Count	720	readings
Minimum	0.0	kWh
Maximum	0.77	kWh
Average	0.57	kWh
Total	406.91	kWh

FINAL ANSWER!

Operational Status Context:

These readings span multiple operational states: **Alarm**, **Idle**, **Maintenance**, **Running**, **Size change**, and **Stopped**. The average is influenced by the mix of these states—notably, the machine spent significant time in **Running** and **Stopped** states, with lower consumption during **Idle** and **Stopped** periods and higher consumption during **Running** operations.